WEB DEVELOPMENT – III

# Node.js Runtime & NPM

*Session 3 — Study Notes*

**Topic: Installing Node.js, Running JS with Node, and NPM**

Instructor: Love Porwal
K.R. Mangalam University — Sohna Road, Gurugram

# 1. Installation of Node.js

A car with a beautifully designed dashboard, leather seats, and a sound system still cannot move an inch without an engine. Everything discussed in the previous two sessions — backend logic, HTTP requests, APIs — is just theory until there is an engine that can actually execute JavaScript outside a browser. Installing Node.js is installing that engine.

Node.js installation brings a JavaScript runtime to your own computer. It is the gateway to backend development — without it, JavaScript is confined to whatever a browser allows, which means only frontend code. Once Node.js is installed, the same language can write servers, command-line tools, and scripts that touch files and networks directly.

Getting it running involves a short, fixed sequence:

1. Download the installer from nodejs.org, choosing the LTS (Long-Term Support) version — the most stable option, recommended for almost all real projects over the newest "Current" release.
2. Run the installer appropriate to the operating system — a .msi file on Windows, a .pkg file on macOS, or a package manager command on Linux.
3. npm (Node Package Manager) installs automatically alongside Node.js — there is no separate step for it.
4. Verify the installation by running node --version and npm --version in a terminal.

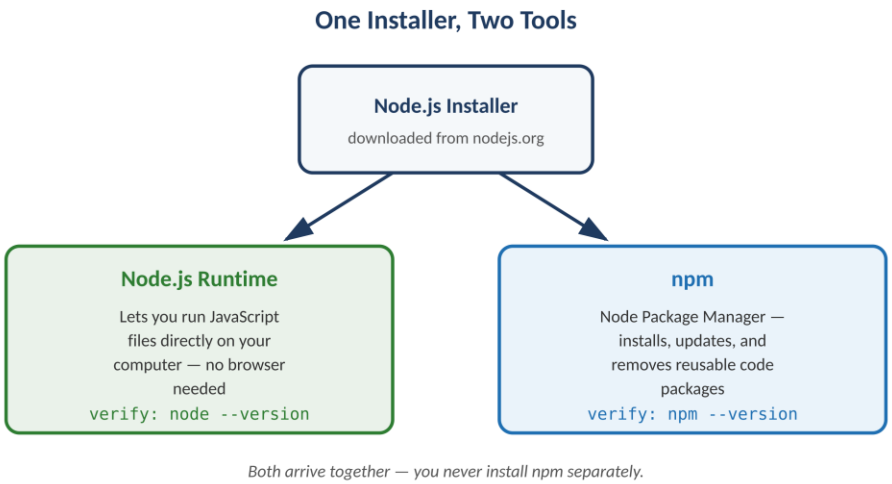


*Figure 1.1 — A single installer, but two tools come out of it.*

Once that verification succeeds, JavaScript is no longer limited to what a browser tab can run. Without Node.js, JavaScript is browser-only — good for buttons and interactivity, but incapable of writing a file, opening a server socket, or connecting to a database. With Node.js installed, the same language can build complete full-stack applications: a React frontend and a Node.js backend, written by the same person, in the same language, without ever switching context.

### Confirming Everything Works

A few commands, run in a terminal right after installation, confirm that both pieces are correctly in place:

```
// Check Node.js version
node --version
// Output: v20.11.0  (your version may differ)

// Check npm version
npm --version
// Output: 10.2.4

// Open the Node.js interactive shell
node
> 2 + 2
4
```

That last block — typing node with no filename — opens the Node.js REPL (Read-Eval-Print Loop), an interactive shell where JavaScript can be typed and run line by line, exactly like a calculator. It's a fast way to test a small snippet without creating a file first.

## 2. Running JavaScript with Node

Beyond the REPL, the far more common way to run JavaScript with Node.js is to save it in a file and hand that file to Node directly. A first script, saved as hello.js, looks like this:

```
// hello.js
console.log('Hello, Node.js!');

// Using a variable
const name = 'Web Development III';
console.log(`Welcome to ${name}`);

// Simple function
function greet(studentName) {
  return `Hello, ${studentName}! Ready to learn Node.js?`;
}

console.log(greet('Alice'));

// Run it from the terminal:
// node hello.js
```

Notice that nothing here looks unfamiliar if JavaScript has already been used in a browser — console.log, template literals, functions, all behave identically. The only difference is where the code runs: instead of a browser tab reading a script tag, the node command reads the file directly off disk and executes it, printing output straight to the terminal instead of a webpage.

## 3. Introduction to NPM

NPM (Node Package Manager) is the world's largest software library for JavaScript, giving access to more than two million reusable code packages. Put simply, npm is a tool used to install and manage JavaScript packages — it lets a developer install code someone else already wrote and tested, update it when a newer version is released, remove it when it's no longer needed, and publish their own packages for others to use.

Without npm, every piece of functionality — reading a file safely, sending an HTTP request, connecting to a database — would have to be written entirely from scratch. npm instead gives instant access to libraries that already solve nearly every common problem. A few concrete examples make this tangible: Express, downloaded more than 50 million times a week, turns "build a web server" into a single command. Mongoose provides a complete toolkit for working with a MongoDB database. Axios handles HTTP requests cleanly, without hand-rolling raw network calls. All three, and millions of others, are one npm install away.

## 4. The NPM Registry

The npm registry is the actual public database where all of these packages live, located at npmjs.com. Anyone — a solo developer, a large company, an open-source community — can publish a package to it, and anyone else can then install that same package with a single command. What gets published there isn't limited to one kind of code; it includes libraries, frameworks, command-line tools, and small utility functions alike.

| Package Name | Use Case |
|---|---|
| Express | Web framework (50M+ weekly downloads) |
| React | UI library |
| nodemon | Auto-restarts the server during development |
| mongoose | MongoDB ODM (object-document mapper) |
| dotenv | Loads environment variables from a file |

## 5. package.json

package.json is a JSON file that sits in a project's root folder and describes the project — its name, version, and every package it depends on. It functions much like a resume for a Node.js project: anyone opening it can immediately see what the project needs to run, without inspecting a single line of actual code.

A simple way to hold this in mind is as a blueprint. A blueprint doesn't contain the building itself, only a precise list of the materials required to construct it. package.json works the same way: it lists every package (ingredient) the application needs, without actually containing that package's code.

Setting one up takes a single command:

```
// Interactive setup — asks questions
npm init
// package name: (my-app)
// version: (1.0.0)
// description: My Node.js API
// entry point: (index.js)
// test command:
// author: Alice
// license: (ISC)

// Skip the questions — use defaults for everything
npm init -y
// Creates package.json instantly with sensible defaults

// Inspect what was created
cat package.json
```

From there, package.json can be edited by hand, but far more often it's updated automatically: running npm install express adds an entry such as "express": "^4.18.2" straight into the dependencies list. Because package.json fully describes what a project needs, sharing just that one file (instead of the entire codebase plus every downloaded package) is enough for someone else to recreate an identical setup — they simply run npm install, and npm downloads everything package.json describes. This is precisely why node_modules, the folder where all that downloaded code actually lives, should never be committed to Git — only package.json and package-lock.json need to travel with the project.

Why this file matters goes beyond convenience. It enables collaboration, since every teammate can install the exact same set of dependencies. It enables deployment, since a production server can rebuild the project from nothing but this file. It supports version management, since it records which package versions the project was built and tested against. And it enables automation, since scripts (like a start or test command) can be defined once here and run identically by anyone.

## 6. Installing Packages

A handful of install commands cover almost everything needed in daily development:

```
// Install and save to 'dependencies' (needed in production)
npm install express
npm install express mongoose dotenv cors
```

```
// Install as a 'devDependency' (development tools only)
npm install --save-dev nodemon jest
// shorthand:
npm i -D nodemon jest

// Install one specific version
npm install express@4.18.2

// Install everything already listed in package.json
npm install

// Install globally (available in any project, as a CLI command)
npm install -g nodemon
```

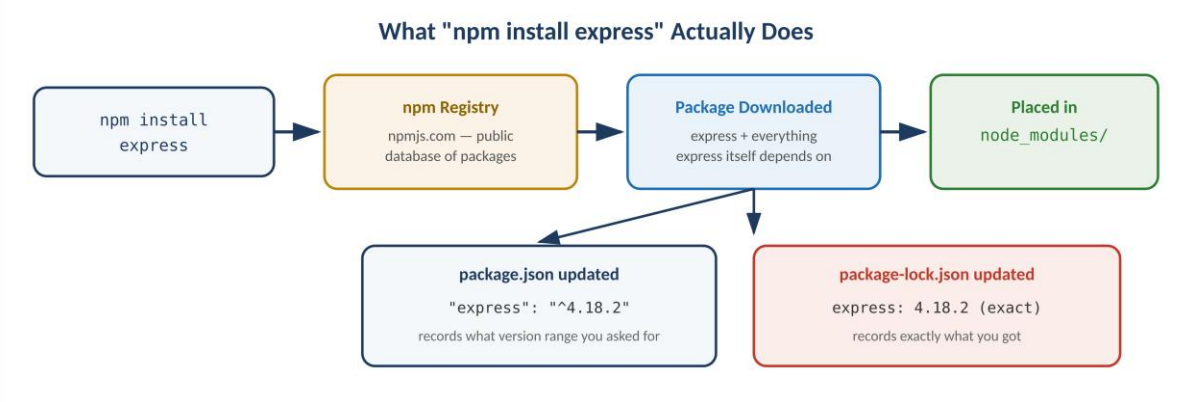


*Figure 6.1 — Following a single npm install command from the registry to your project files.*

The distinction between a regular dependency and a devDependency matters in production. A dependency, such as express, is code the running application actually needs to function. A devDependency, such as jest for testing or nodemon for auto-restarting during development, is only useful while building the project — a production deployment can safely skip installing these, saving space and reducing what's exposed on a live server.

## Local vs Global Packages

Every package installed sits in one of two very different places, and confusing the two is a common source of "why can't my project find this package" bugs.

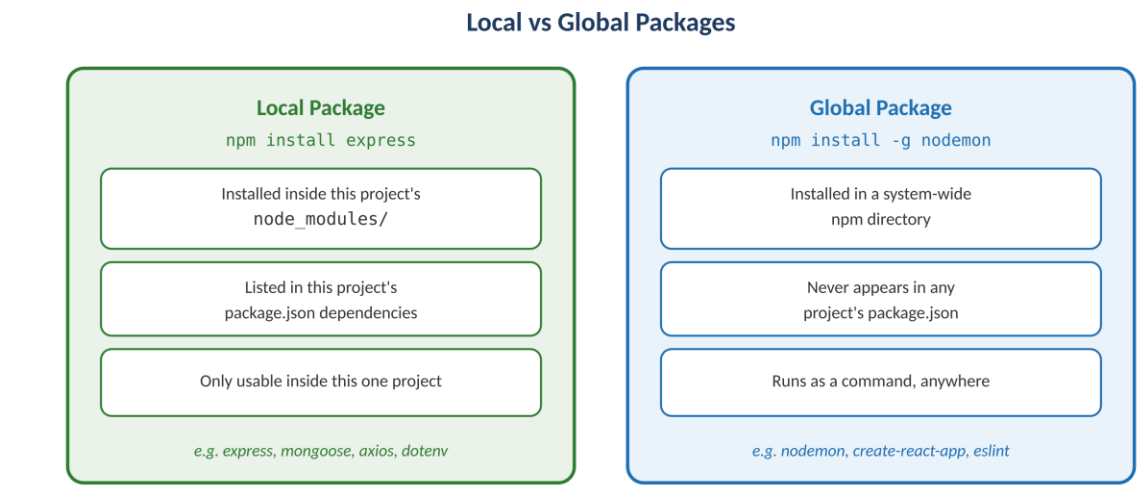


*Figure 6.2 — Two installs that look similar, but live in completely different places.*

A local package — installed with plain npm install <package> — lives inside that project's own node_modules folder, is listed in that project's package.json, and can only be used from within that project. This is where express, mongoose, axios, and dotenv belong, since the application's code directly imports them. A global package — installed with npm install -g <package> — lives in a system-wide npm directory instead, is available as a command in any terminal regardless of which project is open, and never appears in any package.json. Tools like nodemon, create-react-app, or eslint are commonly installed globally, since they're used as commands across many different projects rather than imported into any single one's code.

## 7. package-lock.json

package-lock.json is a file npm generates automatically every time npm install runs — it is never written by hand. It records the exact version of every installed package and, critically, of every sub-dependency those packages themselves rely on, down to the last decimal point.

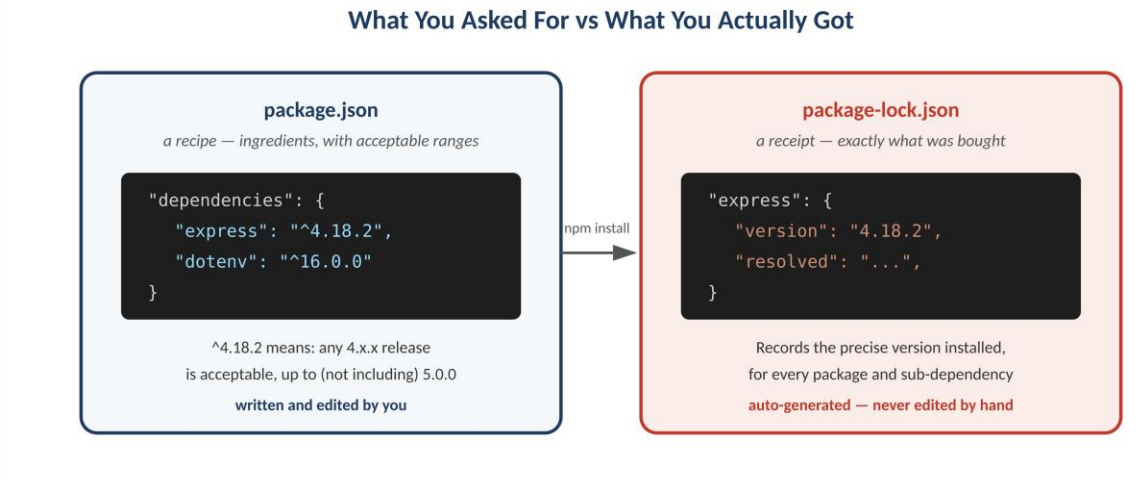


*Figure 7.1 — package.json states a range you'll accept; package-lock.json records exactly what you got.*

The version ranges written in package.json use a small notation that's worth understanding precisely. A caret before a version number, as in "express": "^4.18.2", means npm may install any 4.x.x release from 4.18.2 upward, but never jump to 5.0.0. This exists because version numbers under semantic versioning (SemVer) carry meaning: the first number signals a breaking change, the second signals new backward-compatible features, and the third signals bug fixes. A caret says "give me new features and fixes, but never a breaking change."

That flexibility is exactly the problem package-lock.json solves. Two developers running npm install on the same package.json, weeks apart, could technically end up with different 4.x.x versions of express, if a caret range is the only thing recorded. package-lock.json removes that uncertainty: it pins the precise version actually installed, so that every developer on a team — and the production server itself — installs identical code every single time. This is what prevents the familiar "works on my machine" problem, where a bug appears on one computer only because it happens to be running a slightly different package version than another.

Because of this, package-lock.json should always be committed to Git, never added to .gitignore — it is what makes an entire team's installs reproducible.

## 8. Useful NPM Commands — Quick Reference

| Command | Description |
|---|---|
| npm init -y | Create package.json with defaults |
| npm install <pkg> | Install a package and save it to dependencies |
| npm install -D <pkg> | Install as a devDependency |
| npm install -g <pkg> | Install globally |
| npm uninstall <pkg> | Remove a package |
| npm update | Update all packages within their SemVer ranges |
| npm list | List installed packages in the current project |
| npm list -g | List globally installed packages |
| npm outdated | Check which packages have newer versions available |
| npm run <script> | Run a script defined in package.json |
| npm audit | Check installed packages for known security vulnerabilities |
| npm cache clean --force | Clear the npm cache |

## 9. Key Takeaways

To install a package only for development tools, the correct command is npm install --save-dev package (or its shorthand, npm install -D package). devDependencies are meant strictly for the development phase — testing with jest, auto-restarting with nodemon — and are skipped entirely when deploying to production.

A caret in a version like ^4.18.2 permits any 4.x.x release up to, but never including, 5.0.0. It allows minor feature updates and patch fixes automatically, while protecting the project from an unannounced breaking change that a major version bump (5.0.0) might introduce.

node_modules should never be committed to GitHub, for two independent reasons: it is frequently 100MB or larger, wasting storage and bandwidth for no benefit, and it is entirely redundant — anyone holding package.json can run npm install and regenerate an identical node_modules folder locally. It belongs in .gitignore.

package.json and package-lock.json serve two different, complementary purposes: package.json lists the version ranges a project is willing to accept, while package-lock.json locks in the exact versions that were actually installed. Both should be committed to Git — together, they guarantee that a build is reproducible on any machine, at any time.

## 10. Summary

- Installing Node.js gives a computer two tools at once: the Node.js runtime, which executes JavaScript outside a browser, and npm, which manages packages.
- A .js file is run directly with node filename.js; the REPL (just typing node) runs JavaScript line by line.
- npm is a package manager with access to the npm registry, a public database of over two million reusable JavaScript packages.
- package.json describes a project and its dependencies — a blueprint listing what the project needs, not the code itself.
- Local packages live inside one project's node_modules and are listed in its package.json; global packages install system-wide and run as commands anywhere.
- package-lock.json locks the exact installed versions of every dependency, guaranteeing that every machine ends up with an identical install.

## 11. References


- Node.js Official Documentation — runtime, modules, APIs, file system, HTTP, and Express integration basics.
- W3Schools — Node.js Tutorial: beginner-friendly explanations with hands-on examples.
- freeCodeCamp — Node.js Course: free, project-based learning for Node.js, Express.js, and REST APIs.